WEB DEVELOPMENT · 3RD SEMESTER

Week 06 · Block 6a

The DOM

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Explain what the **DOM** is and how it relates to your HTML
- **Find** elements with `querySelector` using CSS selectors
- **Change** text, attributes, and classes from JavaScript
- **Create**, insert, and remove elements
- **Render** an array of data into the page

Your HTML isn't frozen text.

It's a live tree of objects.

JavaScript can read it, change it, and build new pieces — and the page updates instantly.

PART 1

The DOM

Document Object Model

HTML becomes a tree

When the browser loads your page, it builds the **DOM** — the **Document Object Model**:

```
document
└─ html
   ├─ head
   └─ body
      ├─ h1
      └─ ul → li · li · li
```

- Every element becomes an **object**; the nesting becomes a **tree** *(Block 2a — now live.)*
- `document` is your entry point into it

Live, not a snapshot

- Change an object in the DOM → the browser **re-renders** that part **instantly**
- No reload, no rebuilding the page — you edit the **tree**, the screen follows
- This is exactly what makes a page **interactive**

> The DOM is a two-way mirror: it reflects your HTML, and whatever you change in it is reflected back on screen.

PART 2

Connecting JavaScript

Add a script to the page

```
<head>
  <script src="app.js" type="module"></script>
</head>
```

- Load your JS with **`<script src="…">`**
- Use **`type="module"`** (gives you `import` / `export` from Block 5b — and it runs *after* the page is parsed)
- The old alternatives: **`defer`**, or a plain `<script>` at the **end of** **`<body>`**

Why placement matters: your script must run **after** the elements it wants to touch exist. A module (or `defer`) waits for the DOM automatically.

PART 3

Finding Elements

Query with CSS selectors

```
document.querySelector(".card");      // first match
document.querySelectorAll("li");      // ALL matches (a NodeList)
document.getElementById("title");     // by id, fast
```

- `querySelector` takes a **CSS selector** *(Block 3a!)* → the **first** matching element
- `querySelectorAll` → **all** of them, as a list you can loop
- You already know selectors — now you use them from JavaScript

Loop the matches

```
const items = document.querySelectorAll("li");

for (const item of items) {
  console.log(item.textContent);
}
```

- A `NodeList` is iterable with **`for...of`** *(Block 5b)*
- Grab many elements at once, then act on each

PART 4

Reading & Changing

Text & content

```
const title = document.querySelector("h1");

title.textContent = "Updated!";      // plain text — safe
title.innerHTML = "<em>Hi</em>";     // parses HTML — careful
```

- **`textContent`** — read/write the **text**
- **`innerHTML`** — read/write **HTML** (it parses tags)

⚠️ Never put **user input** into `innerHTML` — that's a cross-site scripting (**XSS**) hole. We tackle it head-on in Block 7a.

Inputs, attributes, classes

```
input.value;                          // what the user typed
img.setAttribute("src", "b.png");     // or:  img.src = "b.png"
el.classList.add("active");           // add a CSS class
el.classList.toggle("open");          // on / off
```

- **`value`** — a form field's current contents
- attributes via `setAttribute` / matching properties (`el.src`, `el.href`)
- **`classList`** — `add` / `remove` / `toggle` a class *(your CSS from Block 3a–3c)*

Style: prefer classes

```
el.style.color = "red";        // inline style — a one-off
el.classList.add("error");     // better: let a CSS class do the styling
```

- `el.style.x` writes an **inline** style — remember, highest specificity *(Block 3b)*
- Prefer **toggling a class** and keeping the look in your CSS
- Keeps **behaviour** (JS) and **appearance** (CSS) separate

PART 5

Creating & Removing

Build new elements

```
const li = document.createElement("li");
li.textContent = "New item";

document.querySelector("ul").append(li);   // insert into the tree
```

- **`createElement`** makes a new, detached element
- set its content / classes, then **`append`** it where you want it
- **`el.remove()`** deletes an element from the page

From data to DOM

```
const fruits = ["apple", "pear", "plum"];
const ul = document.querySelector("ul");

for (const fruit of fruits) {
  const li = document.createElement("li");
  li.textContent = fruit;
  ul.append(li);
}
```

- Loop over **data** → create one element per item → **append**
- This is **rendering**: turning data into DOM — we'll do it constantly *(Block 6b, Block 8)*

Recap

- The **DOM** is your HTML as a live **tree of objects**; `document` is the way in
- Change the DOM → the page **updates instantly**
- **Find** elements with `querySelector(All)` using **CSS selectors**
- **Change** them: `textContent`, `value`, `classList`, attributes *(avoid `innerHTML` with user input)*
- **Create** with `createElement` + `append`; **render** data into the DOM in a loop

<EXERCISE> · 45 MIN

Your turn 👉

Read and reshape a page — all from JavaScript, on load.

1. **Query** elements with `querySelector` / `querySelectorAll`
2. **Change** text, a class, and an attribute
3. **Create** a new element and `append` it
4. **Render** an array of data into a list with a loop

Starter: `exercises/week06/6a/` — open `README.md`. A page and a linked `app.js` run in the browser.